```sql
--Partitioning an existing table using T-SQL
--The steps for partitioning an existing table are as follows:
--Create filegroups
--Create a partition function
--Create a partition scheme
--Create a clustered index on the table based on the partition scheme.
--We'll partition the SinhVien table in the QLHocSinh2 database by NamBD
--Range: <2010; 2011-2020 ; >2020
--Create filegroups
--First, create two new file groups will store the rows with NamBD in 2010 and 2020:
ALTER DATABASE QLHocSinh2
ADD FILEGROUP SinhVien_B2010;

ALTER DATABASE QLHocSinh2
ADD FILEGROUP SinhVien_A2020;

--Second, map the filegroups with the physical files.
ALTER DATABASE QLHocSinh2
ADD FILE     (
    NAME = SinhVien_B2010,
    FILENAME = 'D:\data\SinhVien_B2010.ndf',
```

```
        SIZE = 10 MB,
        MAXSIZE = UNLIMITED,
        FILEGROWTH = 1024 KB
    ) TO FILEGROUP SinhVien_B2010;

ALTER DATABASE QLHocSinh2
ADD FILE     (
    NAME = SinhVien_A2020,
    FILENAME = 'D:\data\SinhVien_A2020.ndf',
        SIZE = 10 MB,
        MAXSIZE = UNLIMITED,
        FILEGROWTH = 1024 KB
    ) TO FILEGROUP SinhVien_A2020;
------------Create a partition function
CREATE PARTITION FUNCTION
Student_by_year_function(int)
AS RANGE LEFT
FOR VALUES (2010, 2020);
--Create a partition scheme
CREATE PARTITION SCHEME Student_by_year_scheme
AS PARTITION Student_by_year_function
TO ([SinhVien_B2010], [primary], SinhVien_A2020);
--Create a clustered index on the partitioning column
--To partition the SinhVien table by the NamBD
column, you
```

```sql
--need to create a clustered index for the NamBD
column on the partition scheme
Student_by_year_scheme.
--change the clustered index that includes the MaSV
column to a non-clustered index NamBD
ALTER TABLE LopHoc
Drop constraint [FK_LopHoc_SinhVien]
GO
ALTER TABLE SinhVien
Drop constraint [PK_SinhVien]
GO

ALTER TABLE SinhVien
ADD PRIMARY KEY NONCLUSTERED([MaSV] ASC)
ON [PRIMARY];

CREATE CLUSTERED INDEX ix_SV_Nam
ON SinhVien
(
    [NamBD]
) ON [Student_by_year_scheme]([NamBD])

ALTER TABLE SinhVien
WITH CHECK ADD FOREIGN KEY([MaLop])
REFERENCES LopHoc([MaLop])
```

```sql
ON UPDATE CASCADE
ON DELETE CASCADE;

ALTER TABLE LopHoc
WITH CHECK ADD FOREIGN KEY([LopTruong])
REFERENCES SinhVien([MaSV])

--To check the number of rows in each partition, you
use the following query:
SELECT
    p.partition_number AS partition_number,
    f.name AS file_group,
    p.rows AS row_count
FROM sys.partitions p
JOIN sys.destination_data_spaces dds ON
p.partition_number = dds.destination_id
JOIN sys.filegroups f ON dds.data_space_id =
f.data_space_id
WHERE OBJECT_NAME(OBJECT_ID) = 'SinhVien'
order by partition_number;

-- Confirm Filegroups
SELECT name as [File Group Name]
FROM sys.filegroups
WHERE type = 'FG'
```

```
GO -- Confirm Datafiles
SELECT name as [DB File Name],physical_name as [DB
File Path]
FROM sys.database_files
where type_desc = 'ROWS'
GO

select * from SinhVien
insert into SinhVien (masv, hoten, MaLop, NamBD)
values ('005',N'Trang','09CLC1',2009)
('004',N'Nam','19CLC1',2019),
('003',N'Nhung','23CLC1',2023),
('002',N'Vy','20CLC1',2020),
('001',N'Hồng','21CLC1',2021)

--select data from a specific partition
SELECT *
FROM dbo.SinhVien
WHERE $PARTITION.Student_by_year_function(NamBD) = 1

SELECT *
FROM dbo.SinhVien
WHERE $PARTITION.Student_by_year_function(NamBD) = 2

SELECT *
```

```
FROM dbo.SinhVien
WHERE $PARTITION.Student_by_year_function(NamBD) = 3

select * from SinhVien
--Get the partition number
SELECT $PARTITION.Student_by_year_function(2010)
```